Manuel de Référence Théorique: Architecture des Bases de Données

ACID (Atomicité, Consistance, Isolation, Durabilité)

Signification Sémantique: Le paradigme ACID définit un ensemble de propriétés transactionnelles garantissant que les opérations sur les Data s'exécutent de manière fiable, même en cas d'erreur ou de défaillance matérielle. **Simplified Version**: Une garantie stricte qui assure que les modifications apportées aux informations sont soit complètement enregistrées, soit totalement ignorées en cas de problème technique. **Frontières Théoriques**: ACID ne garantit pas la sécurité contre les accès non autorisés ni les performances d'exécution des Query. ACID ne s'applique qu'aux Transaction et non au Database Schema lui-même. **Rôle Architectural**: Le paradigme ACID agit comme un contrat de fiabilité au sein du Relational Database Management System, assurant la cohérence globale de l'état du Database.

Association

Signification Sémantique: Une Association représente un lien sémantique et structurel établi entre deux ou plusieurs Entity Class au sein d'un Data Model. **Simplified Version**: Une connexion formelle définie entre différentes catégories d'informations pour montrer comment ces catégories interagissent ou dépendent les unes des autres. **Frontières Théoriques**: L'Association n'est pas une Entity en soi (sauf dans le cas d'une Associative Entity). L'Association ne stocke pas de Data intrinsèques au-delà des clés de liaison. **Rôle Architectural**: L'Association structure la topologie relationnelle, permettant la navigation inter-entités et la résolution des Foreign Key lors de l'implémentation physique. **Illustration Structurelle**:

```sql
CREATE TABLE Order (
    OrderID INT PRIMARY KEY,
    UserID INT,
    FOREIGN KEY (UserID) REFERENCES User(UserID)
);
```

Association Cardinality (One-to-One, One-to-Many, Many-to-Many)

Signification Sémantique: L'Association Cardinality définit les limites quantitatives strictes d'instances d'une Entity Class pouvant participer à une Association avec les instances d'une autre Entity Class. **Simplified Version**: Les règles numériques qui précisent exactement combien d'éléments d'une catégorie peuvent être reliés à un élément d'une autre catégorie. **Frontières Théoriques**: L'Association Cardinality ne définit pas la nature du lien ni la chronologie des insertions de Data, mais uniquement les contraintes de multiplicité. **Rôle Architectural**: L'Association Cardinality détermine l'architecture du Relational Schema, dictant la création de tables de jonction pour le Many-to-Many ou le placement de Foreign Key pour le One-to-Many.

Association récursive

Signification Sémantique: Une Recursive Association est une configuration structurelle où une Entity Class participe à une Association avec elle-même. **Simplified Version**: Un lien où un élément d'une catégorie est connecté à un autre élément appartenant exactement à la même catégorie. **Frontières Théoriques**: La Recursive Association n'implique pas de boucle infinie de Data. La Recursive Association nécessite une gestion explicite des Roles in Association pour distinguer les entités participantes. **Rôle Architectural**: La Recursive Association modélise les structures hiérarchiques ou les graphes au sein d'une même Table, utilisant souvent une Foreign Key qui référence la Primary Key de la même Table. **Illustration Structurelle**:

```sql
CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    ManagerID INT,
    FOREIGN KEY (ManagerID) REFERENCES Employee(EmployeeID)
);
```

Association ternaire

Signification Sémantique: Une Ternary Association est une relation simultanée et indivisible liant exactement trois Entity Class distinctes. **Simplified Version**: Une connexion complexe qui exige la participation simultanée de trois éléments différents pour que le lien ait un sens. **Frontières Théoriques**: La Ternary Association ne peut pas être décomposée en trois Binary Association indépendantes sans perte de sémantique. **Rôle Architectural**: La Ternary Association se traduit au niveau du Relational Schema par une Table associative dédiée contenant au minimum les trois Foreign Key des Entity Class impliquées.

Attribut

Signification Sémantique: Un Attribute est une propriété nommée qui définit une caractéristique scalaire ou structurelle d'une Entity Class ou d'une Association. **Simplified Version**: Une donnée spécifique et individuelle qui décrit une particularité d'une catégorie d'informations. **Frontières Théoriques**: Un Attribute ne contient pas d'entités complexes ni de relations. Un Attribute représente la plus petite unité d'information gérée par le Data Model. **Rôle Architectural**: L'Attribute définit les Column au sein du Relational Schema, déterminant le Data Type et les contraintes de stockage de l'information.

Attribut atomique

Signification Sémantique: Un Atomic Attribute est un Attribute dont la valeur est conceptuellement indivisible et ne peut être fragmentée en sous-composants sémantiques. **Simplified Version**: Une information élémentaire qui ne peut pas être découpée en morceaux plus petits sans perdre tout son sens. **Frontières Théoriques**: L'Atomic Attribute ne peut pas contenir de sous-attributs. L'Atomic Attribute s'oppose directement au Composite Attribute. **Rôle Architectural**: L'Atomic Attribute correspond directement à une Column unique et simple dans le Physical Schema du Database.

Attribut composite

Signification Sémantique: Un Composite Attribute est un Attribute qui est lui-même constitué d'une hiérarchie d'autres Attribute plus fondamentaux. **Simplified Version**: Une information complexe qui est construite en regroupant plusieurs informations plus simples. **Frontières Théoriques**: Le Composite

Attribute n'est pas une Entity Class séparée, car il n'a pas d'existence indépendante de l'entité qui le porte. **Rôle Architectural**: Dans un Relational Database System, le Composite Attribute doit généralement être aplati en multiples Atomic Attribute distincts lors de la phase de Logical Design.

Attribut dérivé

Signification Sémantique: Un Derived Attribute est un Attribute dont la valeur n'est pas physiquement stockée mais est calculée dynamiquement à partir d'autres Attribute ou entités. **Simplified Version**: Une information qui n'est pas sauvegardée directement, mais qui est déduite ou calculée à la volée à partir d'autres données existantes. **Frontières Théoriques**: Le Derived Attribute ne fait pas partie de la source de vérité persistante. Le Derived Attribute dépend entièrement de l'état des autres Attribute du Database. **Rôle Architectural**: Le Derived Attribute est souvent implémenté via des View, des Trigger, ou calculé au sein de la Query via le Data Manipulation Language pour éviter la Data Redundancy.

Attribut identifiant

Signification Sémantique: Un Identifying Attribute est un Attribute, ou un ensemble d'Attribute, dont la valeur distingue de manière unique et univoque chaque instance au sein d'une Entity Class. **Simplified Version**: Une information spécifique qui sert de plaque d'immatriculation pour garantir qu'aucun élément ne puisse être confondu avec un autre. **Frontières Théoriques**: L'Identifying Attribute ne doit jamais être soumis à la Null Value et sa valeur ne doit idéalement jamais muter au cours du cycle de vie de l'entité. **Rôle Architectural**: L'Identifying Attribute devient la Primary Key lors de la traduction du Conceptual Schema vers le Logical Schema.

Base de données

Signification Sémantique: Un Database est une collection structurée, persistante et interdépendante de Data, organisée selon un Data Model spécifique pour permettre l'interrogation et la manipulation systématiques. **Simplified Version**: Un grand classeur électronique ultra-organisé conçu pour stocker, retrouver et modifier des informations de manière extrêmement efficace. **Frontières Théoriques**: Le Database représente le conteneur passif de l'information et ne doit pas être confondu avec le Database Management System, qui est le logiciel actif contrôlant l'accès à ce conteneur. **Rôle Architectural**: Le Database sert de couche de persistance centrale dans les architectures logicielles, découplant le stockage de l'information de la Business Logic des applications.

Cardinalité

Signification Sémantique: La Cardinality est une contrainte quantitative définissant le nombre minimum et maximum d'occurrences d'une Entity participant à une relation structurelle. **Simplified Version**: Les limites mathématiques qui définissent le nombre minimal et maximal de liens autorisés pour un élément précis. **Frontières Théoriques**: La Cardinality se limite à la quantification topologique. La Cardinality ne définit pas le Data Type des entités liées ni les règles d'intégrité internes. **Rôle Architectural**: La Cardinality guide les règles de transformation du Conceptual Model vers le Relational Model, influençant l'application de contraintes de non-nullité et la génération de Table supplémentaires.

Clé étrangère (FK)

Signification Sémantique: Une Foreign Key est un Attribute ou un ensemble d'Attribute dans une Table qui référence de manière contrainte la Primary Key d'une autre (ou de la même) Table. **Simplified Version**: Une référence sécurisée qui pointe vers l'identifiant exact d'une information située dans une autre liste pour maintenir les deux listes connectées. **Frontières Théoriques**: La Foreign Key ne garantit pas l'unicité de ses propres valeurs au sein de sa Table hôte, sauf si elle est également désignée comme Primary Key. **Rôle Architectural**: La Foreign Key implémente le Referential Integrity constraint au sein du Physical Schema, empêchant la création de liaisons orphelines et orchestrant les suppressions en cascade. **Illustration Structurelle**:

```
ALTER TABLE ChildTable
ADD CONSTRAINT FK_Parent
FOREIGN KEY (ParentID) REFERENCES ParentTable(PrimaryID);
```

Clé primaire (PK)

Signification Sémantique: Une Primary Key est la désignation formelle d'un ou plusieurs Attribute garantissant l'unicité et la non-nullité absolues pour chaque Record au sein d'une Table. **Simplified Version**: L'identifiant principal et exclusif d'une information, garantissant que chaque élément enregistré possède une identité propre et non vide. **Frontières Théoriques**: La Primary Key ne peut admettre aucune Null Value. La Primary Key est immuable d'un point de vue architectural pour éviter de briser la Referential Integrity. **Rôle Architectural**: La Primary Key sert de point d'ancrage fondamental pour l'indexation physique (souvent un Clustered Index) et constitue la cible unique des Foreign Key.

Contrainte d'intégrité (Primary Key, Foreign Key, Domain, Line, Column)

Signification Sémantique: Une Integrity Constraint est une règle déclarative imposée sur le Relational Schema pour garantir que toutes les Data insérées ou modifiées respectent les assertions sémantiques et structurelles du domaine. **Simplified Version**: Un ensemble de règles de sécurité strictes empêchant l'insertion de données invalides, contradictoires ou incomplètes dans le système. **Frontières Théoriques**: L'Integrity Constraint ne gère pas la Business Logic applicative complexe. L'Integrity Constraint agit comme un garde-fou passif au niveau du Storage Engine. **Rôle Architectural**: L'Integrity Constraint maintient la validité de l'état du Database au niveau de la Column (Domain Constraint) ou entre les Table (Referential Constraint), rejetant toute Transaction violant ces règles.

Couche conceptuelle

Signification Sémantique: Le Conceptual Schema est une abstraction de haut niveau décrivant la sémantique, les entités et les relations du domaine d'information, indépendamment de toute considération technologique. **Simplified Version**: Le plan d'architecte initial qui décrit ce que le système doit retenir et comment les idées sont liées, sans se soucier du logiciel qui sera utilisé. **Frontières Théoriques**: Le Conceptual Schema ignore totalement les contraintes de performance, le Database Management System cible, et les détails de persistance comme les Data Type physiques. **Rôle Architectural**: Le Conceptual Schema sert d'interface de communication entre les Domain Expert et les Data Architect, fournissant la fondation pour la dérivation du Logical Schema.

Couche logique / Community Schema

Signification Sémantique: Le Logical Schema représente la traduction du Conceptual Schema en une structure de données mathématiquement rigoureuse, normalisée et prête à être implémentée, mais toujours indépendante du matériel. **Simplified Version**: Le plan technique détaillé qui organise les informations en tableaux et colonnes formels, préparant le terrain pour l'ordinateur. **Frontières Théoriques**: Le Logical Schema définit les Primary Key et les Foreign Key, mais ne se préoccupe pas de l'allocation des disques ni des Index d'optimisation. **Rôle Architectural**: Le Logical Schema définit la structure globale partagée par l'ensemble des applications (Community Schema), servant de base pour l'écriture du Data Definition Language.

Couche physique / Internal Schema

Signification Sémantique: Le Physical Schema définit l'implémentation concrète du Logical Schema sur le Database Management System spécifique, dictant l'allocation spatiale, le partitionnement et l'indexation. **Simplified Version**: La configuration technique ultime qui dicte exactement comment et où les fichiers sont physiquement écrits et optimisés sur le disque dur de l'ordinateur. **Frontières Théoriques**: Le Physical Schema n'altère pas la sémantique ni les relations définies par le Logical Schema. Le Physical Schema est hautement dépendant de la technologie propriétaire. **Rôle Architectural**: Le Physical Schema optimise les entrées/sorties et les performances d'exécution des Query, isolant ces détails techniques du niveau applicatif via le principe d'indépendance physique des données.

Data catalog

Signification Sémantique: Un Data Catalog est un inventaire organisationnel exhaustif qui utilise des Metadata pour faciliter la découverte, la gouvernance et la compréhension des Data Asset au sein d'une entreprise. **Simplified Version**: Un moteur de recherche interne et un répertoire détaillé permettant aux équipes de trouver rapidement quelles informations existent et à quoi elles servent. **Frontières Théoriques**: Le Data Catalog ne stocke pas les Data primaires elles-mêmes. Le Data Catalog s'intéresse à la gouvernance et au lignage de l'information, et non à l'exécution des Query. **Rôle Architectural**: Le Data Catalog agit comme une surcouche de gestion par-dessus de multiples Database et Data Warehouse, centralisant l'accès aux Metadata descriptives.

Data dictionary

Signification Sémantique: Le Data Dictionary est le composant interne du Database Management System qui stocke la définition formelle de toutes les structures (Table, View, Index) et des contraintes du Relational Schema. **Simplified Version**: Le manuel de référence interne du système qui documente la structure exacte de chaque tableau, colonne et règle de sécurité. **Frontières Théoriques**: Le Data Dictionary est spécifique à une instance de Database unique, contrairement au Data Catalog. Le Data Dictionary est géré automatiquement par le système lors de l'exécution d'instructions Data Definition Language. **Rôle Architectural**: Le Data Dictionary est continuellement consulté par le Query Optimizer et le moteur d'exécution pour valider la syntaxe, les permissions et résoudre les chemins d'accès aux données.

Database Lifecycle (Analysis, Modeling, Implementation, Test/Evaluation, Use)

Signification Sémantique: Le Database Lifecycle est le cadre méthodologique séquentiel dictant la conception, la création, le déploiement, et la maintenance opérationnelle d'un système d'information. **Simplified Version**: Le cycle de vie complet qui décrit toutes les étapes de fabrication et d'utilisation d'un

système de stockage, depuis l'idée initiale jusqu'à son fonctionnement quotidien. **Frontières Théoriques**: Le Database Lifecycle n'est pas un algorithme informatique, mais un processus d'ingénierie logicielle. Le Database Lifecycle se concentre sur l'évolution temporelle du Database System. **Rôle Architectural**: Le Database Lifecycle orchestre la transition de l'abstraction vers la concrétisation, assurant que l'architecture répond aux exigences métier.

DB-Main

Signification Sémantique: DB-Main est un outil d'ingénierie de modélisation de données spécialisé dans la conception, la rétro-ingénierie et l'évolution des Database Schema selon des méthodologies formelles. **Simplified Version**: Un logiciel spécialisé utilisé par les concepteurs pour dessiner les plans techniques des systèmes d'information et générer automatiquement le code correspondant. **Frontières Théoriques**: DB-Main n'est pas un Database Management System. DB-Main ne stocke ni ne gère de Data opérationnelles, il gère uniquement des abstractions et des Metadata. **Rôle Architectural**: DB-Main intervient dans la phase de Modeling du Database Lifecycle, permettant la traduction automatisée d'un Conceptual Schema vers un Logical Schema puis un Physical Schema.

DBMS / SGBD

Signification Sémantique: Un Database Management System est la suite logicielle complexe qui sert d'interface intermédiaire entre l'utilisateur, les applications et le Database physique pour garantir le stockage, la récupération et la sécurité des données. **Simplified Version**: Le programme maître ultra-puissant qui gère toutes les demandes de lecture ou d'écriture, s'assurant que l'accès aux informations est rapide, sûr et organisé. **Frontières Théoriques**: Le Database Management System n'est pas le Database lui-même. Le Database Management System n'est pas responsable de la Business Logic de l'application cliente. **Rôle Architectural**: Le Database Management System agit comme le moteur d'exécution exclusif, imposant le paradigme ACID, gérant la concurrence, et traduisant les requêtes Data Manipulation Language en opérations disque.

DDL (Data Definition Language)

Signification Sémantique: Le Data Definition Language est le sous-ensemble de syntaxes SQL dédié à la création, la modification et la destruction des structures architecturales et des contraintes du Database Schema. **Simplified Version**: Le langage de commande utilisé pour construire, modifier ou détruire les fondations et les murs du système de stockage, sans toucher aux objets à l'intérieur. **Frontières Théoriques**: Le Data Definition Language n'interagit pas avec les Data elles-mêmes. Le Data Definition Language manipule exclusivement les Metadata contenues dans le Data Dictionary. **Rôle Architectural**: Le Data Definition Language matérialise le Logical Schema et le Physical Schema au sein du Database Management System. **Illustration Structurelle**:

```sql
CREATE TABLE Product (
    ProductID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL
);
```

DML (Data Manipulation Language)

Signification Sémantique: Le Data Manipulation Language est le sous-ensemble de syntaxes SQL permettant l'insertion, la mise à jour, et la suppression des Data opérationnelles à l'intérieur des structures existantes. **Simplified Version**: Le langage de commande utilisé exclusivement pour ajouter, modifier ou effacer les informations stockées dans les tableaux préexistants. **Frontières Théoriques**: Le Data Manipulation Language ne peut altérer ni le Relational Schema ni les objets du Database. Le Data Manipulation Language est strictement confiné à la manipulation des Record. **Rôle Architectural**: Le Data Manipulation Language constitue l'interface d'écriture transactionnelle principale utilisée par la Business Logic de l'application pour modifier l'état du Database.

Donnée persistante

Signification Sémantique: La Persistent Data est une donnée dont la durée de vie dépasse l'exécution du processus informatique qui l'a créée, survivant aux redémarrages du système via un stockage sur mémoire non volatile. **Simplified Version**: Une information qui est sauvegardée de manière permanente sur un disque dur afin de ne jamais être perdue lorsque l'ordinateur s'éteint. **Frontières Théoriques**: La Persistent Data n'est pas stockée dans la Random Access Memory. La Persistent Data implique une pénalité de performance due aux opérations d'écriture sur le disque. **Rôle Architectural**: La Persistent Data constitue l'état fondamental du système d'information, garanti par la propriété de Durability du paradigme ACID au sein du Database Management System.

Donnée transitoire

Signification Sémantique: La Transient Data est une donnée éphémère maintenue exclusivement dans la mémoire volatile durant l'exécution d'un processus et qui est irrémédiablement détruite à l'arrêt de ce dernier. **Simplified Version**: Une information temporaire conservée dans la mémoire vive juste le temps d'effectuer un calcul ou une tâche, puis définitivement oubliée. **Frontières Théoriques**: La Transient Data ne survit pas aux pannes de courant ou aux redémarrages de l'application. La Transient Data n'est soumise à aucune propriété ACID. **Rôle Architectural**: La Transient Data est utilisée pour le traitement algorithmique local, les variables d'application et les caches temporaires avant toute transformation en Persistent Data.

DRL (Data Retrieval Language)

Signification Sémantique: Le Data Retrieval Language est la composante syntaxique spécialisée dans l'interrogation, le filtrage et l'extraction de sous-ensembles de Data sans en altérer l'état. **Simplified Version**: Le langage de commande utilisé uniquement pour poser des questions au système et récupérer des listes d'informations sans rien modifier. **Frontières Théoriques**: Le Data Retrieval Language garantit l'immutabilité des Data lors de son exécution. Le Data Retrieval Language ne définit ni ne modifie la structure de la base. **Rôle Architectural**: Le Data Retrieval Language s'interface avec le Query Optimizer pour naviguer dans le Physical Schema et retourner des Projection ou des Selection à l'application consommatrice. **Illustration Structurelle**:

```
SELECT Name, Price
FROM Product
WHERE Price > 100;
```

Entity / Entité

Signification Sémantique: Une Entity est une instance unique, distincte et identifiable existant dans le domaine d'information, caractérisée par un ensemble d'Attribute. **Simplified Version**: Un élément ou un objet précis et unique du monde réel que l'on souhaite représenter et mémoriser dans le système. **Frontières Théoriques**: L'Entity représente une occurrence singulière et non la catégorie abstraite (qui est l'Entity Class). **Rôle Architectural**: L'Entity correspond à un unique Record persistant à l'intérieur d'une Table dans le Relational Database System.

Entity Class / Classe d'Entité

Signification Sémantique: Une Entity Class est une abstraction modélisant un ensemble catégorique d'Entity partageant la même structure d'Attribute et la même sémantique. **Simplified Version**: Le modèle générique ou le moule qui définit la structure commune pour un ensemble d'objets du même type. **Frontières Théoriques**: L'Entity Class définit le schéma conceptuel mais ne contient pas les données individuelles elles-mêmes. L'Entity Class est purement au niveau du Logical Design. **Rôle Architectural**: L'Entity Class est traduite architecturalement par une Table ou une Relation lors du passage au Relational Schema, définissant la structure d'accueil pour les futurs Record.

External level / Couche externe

Signification Sémantique: L'External Schema définit la vue d'abstraction la plus haute du Database, présentant uniquement les sous-ensembles de Data pertinents et sécurisés pour un utilisateur ou une application spécifique. **Simplified Version**: L'affichage sur mesure qui ne montre à un utilisateur spécifique que les informations auxquelles il a le droit d'accéder, cachant toute la complexité sous-jacente. **Frontières Théoriques**: L'External Schema ne dicte en rien la manière dont les Data sont physiquement stockées ni structurées logiquement de manière globale. **Rôle Architectural**: L'External Schema garantit le principe d'indépendance logique des données et est implémenté techniquement au travers de View ou de procédures stockées dédiées.

Héritage

Signification Sémantique: L'Inheritance est un mécanisme structurel par lequel une Subclass dérive automatiquement des Attribute et des Association définis par une Superclass, permettant une hiérarchie sémantique. **Simplified Version**: Un mécanisme de transmission où une sous-catégorie récupère automatiquement toutes les caractéristiques de la catégorie parente tout en pouvant y ajouter les siennes. **Frontières Théoriques**: L'Inheritance dans le Database Modeling décrit les données statiques, et non les comportements algorithmiques comme dans la Programmation Orientée Objet. **Rôle Architectural**: L'Inheritance se traduit dans le Relational Schema par des architectures spécifiques telles que le Class Table Pattern ou le Single Table Pattern pour maintenir la cohérence de la hiérarchie.

Information structure (Data, Information, Knowledge, Wisdom)

Signification Sémantique: La pyramide DIKW définit le continuum théorique de l'abstraction où les signaux bruts (Data) sont contextualisés (Information), puis synthétisés en règles (Knowledge) et enfin appliqués avec jugement (Wisdom). **Simplified Version**: La progression théorique expliquant comment de simples chiffres bruts sont transformés étape par étape en une compréhension profonde permettant de prendre de bonnes décisions. **Frontières Théoriques**: Le Database Management System classique ne gère fondamentalement que la couche Data et la couche Information. Les couches supérieures relèvent de

l'analyse décisionnelle. **Rôle Architectural**: Cette hiérarchie justifie la transition architecturale entre le Transactional Database pour les Data et les environnements de Data Warehouse pour le Knowledge.

Intégrité référentielle

Signification Sémantique: La Referential Integrity est une contrainte architecturale absolue garantissant que toute valeur d'une Foreign Key pointant vers une autre Table doit correspondre à une Primary Key existante dans cette Table cible. **Simplified Version**: La règle de sécurité qui empêche strictement de faire référence à un élément qui n'existe pas ou qui a été supprimé. **Frontières Théoriques**: La Referential Integrity ne garantit pas l'exactitude des valeurs elles-mêmes, mais uniquement la validité topologique des liens entre les Table. **Rôle Architectural**: La Referential Integrity est appliquée activement par le Database Management System pour prévenir les données orphelines, rejetant le Data Manipulation Language invalide ou exécutant des Cascading Deletes.

Métadonnée

Signification Sémantique: La Metadata est une donnée auto-descriptive qualifiant et définissant la structure, la nature, la provenance ou la sémantique d'autres Data au sein du système. **Simplified Version**: L'information qui décrit l'information, comme une étiquette sur une boîte indiquant ce qu'il y a à l'intérieur et quand elle a été emballée. **Frontières Théoriques**: La Metadata n'est pas l'information opérationnelle finale consommée par l'application métier. La Metadata décrit le conteneur plutôt que le contenu. **Rôle Architectural**: La Metadata peuple le Data Dictionary et permet au Database Management System, ainsi qu'au Data Catalog, d'orchestrer la gouvernance, l'indexation et la vérification des types de données.

Modèle entité-association

Signification Sémantique: L'Entity-Relationship Model est un métamodèle sémantique formel utilisé lors du Conceptual Design pour représenter l'univers du discours au travers d'Entity Class, d'Attribute et d'Association. **Simplified Version**: Un standard de dessin technique utilisé pour cartographier visuellement les différentes catégories d'informations et les liens logiques qui les unissent. **Frontières Théoriques**: L'Entity-Relationship Model est agnostique vis-à-vis de la technologie de stockage sous-jacente. L'Entity-Relationship Model ne définit ni les Index ni les Data Type physiques. **Rôle Architectural**: L'Entity-Relationship Model sert de pivot d'abstraction fondamental, guidant la transformation rigoureuse des besoins métier vers la création de Table relationnelles.

Modélisation de données

Signification Sémantique: Le Data Modeling est l'ingénierie analytique consistant à abstraire un système d'information complexe en schémas formels consécutifs pour garantir l'intégrité et la normalisation des Data. **Simplified Version**: Le processus intellectuel et technique de traduction des règles de l'entreprise en une architecture de stockage informatique solide et sans faille. **Frontières Théoriques**: Le Data Modeling ne concerne pas le codage de l'interface utilisateur ou du comportement applicatif. Le Data Modeling se focalise exclusivement sur la structure de l'information. **Rôle Architectural**: Le Data Modeling prévient la Data Redundancy et l'anomalie transactionnelle en appliquant les règles de la Database Normalization avant toute implémentation technique.

Multiple Inheritance

Signification Sémantique: Le Multiple Inheritance est une construction hiérarchique complexe où une unique Subclass dérive simultanément des Attribute et Association de plusieurs Superclass distinctes. **Simplified Version**: Une situation complexe où une sous-catégorie hérite directement des caractéristiques de plusieurs catégories parentes différentes en même temps. **Frontières Théoriques**: Le Multiple Inheritance introduit des risques d'ambiguïté de résolution. Le Multiple Inheritance est souvent rejeté par les Relational Database Management System stricts. **Rôle Architectural**: Le Multiple Inheritance requiert des stratégies d'aplatissement complexes lors de la traduction vers le Relational Schema, nécessitant souvent des Table de jonction sophistiquées.

Multiple Specialization

Signification Sémantique: La Multiple Specialization définit le partitionnement simultané d'une Superclass selon plusieurs critères de classification indépendants, générant des grappes distinctes de Subclass. **Simplified Version**: Le découpage d'une grande catégorie mère en plusieurs groupes de sous-catégories distincts, en utilisant plusieurs règles de tri différentes en parallèle. **Frontières Théoriques**: La Multiple Specialization se distingue du Multiple Inheritance car la divergence s'opère du parent vers les enfants, et non l'inverse. Les critères de spécialisation restent sémantiquement orthogonaux. **Rôle Architectural**: La Multiple Specialization impose la création de Discriminator Column multiples dans le Physical Schema pour router correctement l'instanciation au sein du système.

Redondance

Signification Sémantique: La Data Redundancy est la duplication non nécessaire de la même information persistante en de multiples emplacements distincts au sein du Database Schema. **Simplified Version**: Le fait d'enregistrer la même information à plusieurs endroits différents, ce qui gaspille de la place et risque de créer des contradictions. **Frontières Théoriques**: La Data Redundancy diffère de la réplication physique légitime utilisée pour la haute disponibilité. La Data Redundancy désigne ici une faille de conception logique. **Rôle Architectural**: La Data Redundancy induit des anomalies de mise à jour et de suppression, poussant le Data Architect à appliquer le processus de Database Normalization pour l'éliminer.

Relation is-a

Signification Sémantique: L'Is-A Relationship est la traduction sémantique fondamentale de l'Inheritance, déclarant qu'une instance de la Subclass est conceptuellement une instance parfaitement valide de la Superclass. **Simplified Version**: Le principe logique affirmant qu'un élément d'une sous-catégorie appartient pleinement et de droit à la catégorie générale parente. **Frontières Théoriques**: L'Is-A Relationship est strictement asymétrique et transitive. L'Is-A Relationship ne décrit pas l'assemblage conceptuel. **Rôle Architectural**: L'Is-A Relationship justifie la propagation de la Primary Key de la Table parente comme Primary Key et Foreign Key simultanément dans la Table enfant au sein du Physical Schema.

Roles in Association

Signification Sémantique: Les Roles in Association sont des identifiants sémantiques attribués aux terminaisons d'une Association pour désambiguïser la nature de l'implication de chaque Entity Class, particulièrement indispensable lors d'une Recursive Association. **Simplified Version**: Des étiquettes précises placées sur un lien pour expliquer exactement quel rôle joue chaque élément par rapport à l'autre. **Frontières Théoriques**: Le Role n'est pas une Entity ni un Attribute. Le Role sert exclusivement de

clarificateur sémantique dans le Conceptual Model. **Rôle Architectural**: Le Role influence la nomenclature finale des Foreign Key dans le Logical Schema pour garantir la clarté du Data Definition Language généré.

Schéma relationnel

Signification Sémantique: Le Relational Schema est l'ensemble mathématiquement défini des structures de Table, de leurs Column respectives et des Integrity Constraint, exprimé selon les postulats de l'algèbre relationnelle. **Simplified Version**: Le plan technique final et formel constitué uniquement de tableaux, de colonnes et de liens, prêt à être codé dans le système informatique. **Frontières Théoriques**: Le Relational Schema exclut toute abstraction objet pure et requiert une traduction stricte en relations tabulaires planes. **Rôle Architectural**: Le Relational Schema est l'artefact direct à partir duquel les scripts SQL Data Definition Language sont générés pour construire la base opérationnelle finale.

Spécialisation-Généralisation

Signification Sémantique: La Specialization-Generalization est l'axe d'abstraction bidirectionnel régissant l'Inheritance. La Spécialisation crée des Subclass détaillées à partir d'une Superclass, tandis que la Généralisation factorise les Attribute communs de plusieurs Entity Class vers une Superclass unifiée. **Simplified Version**: Le mouvement logique de conception consistant soit à détailler une grande catégorie en sous-groupes spécifiques, soit à regrouper plusieurs petits groupes sous un grand concept commun. **Frontières Théoriques**: Ce concept est purement analytique et sémantique. Au niveau du Physical Schema, aucune distinction architecturale n'existe entre la conception par spécialisation ou par généralisation. **Rôle Architectural**: La Specialization-Generalization permet l'optimisation conceptuelle en éliminant la Data Redundancy dans le modèle abstrait en regroupant les Attribute factorisables.

Specialization Cardinality (Total/Partial, Exclusive/Overlapping)

Signification Sémantique: La Specialization Constraint définit l'exhaustivité et la disjonction des Subclass par rapport à leur Superclass dans une hiérarchie d'héritage. **Simplified Version**: Les règles strictes qui décident si un élément parent doit obligatoirement appartenir à une sous-catégorie, et s'il a le droit d'appartenir à plusieurs sous-catégories à la fois. **Frontières Théoriques**: Ces contraintes s'appliquent exclusivement à l'Is-A Relationship et n'ont aucune influence sur les Association standards entre Entity Class distinctes. **Rôle Architectural**: La Specialization Constraint impacte radicalement l'implémentation logique. Une hiérarchie exclusive force l'utilisation de Check Constraint stricts, tandis qu'une hiérarchie superposée peut exiger de multiples Foreign Key.

SQL

Signification Sémantique: SQL est le langage déclaratif standardisé de domaine spécifique conçu pour la gestion, la définition, le contrôle et l'interrogation algorithmique des Data au sein d'un Relational Database Management System. **Simplified Version**: Le langage de programmation standard et universel utilisé pour donner des ordres directs à la base de données afin de manipuler sa structure ou ses informations. **Frontières Théoriques**: SQL est syntaxiquement déclaratif et spécifie le résultat attendu et non la procédure d'accès séquentielle. SQL n'est théoriquement pas un langage généraliste Turing-complet sans extensions procédurales. **Rôle Architectural**: SQL fait office d'interface universelle de communication textuelle entre l'application cliente et le Query Parser du Database Management System.

Système de gestion de bases de données (SGBD)

Signification Sémantique: Un Database Management System est le noyau d'infrastructure logicielle régissant la persistance structurée, la concurrence, la sécurité et la cohérence de l'information sous la garantie ACID. **Simplified Version**: L'application logicielle de fond de très haute performance qui contrôle de manière absolue tout ce qui entre et sort de la base de données. **Frontières Théoriques**: Le Database Management System n'est pas le système d'exploitation et n'est pas responsable du Front-End applicatif. **Rôle Architectural**: Le Database Management System encapsule le Physical Schema, fournissant une abstraction d'accès via le Data Manipulation Language tout en optimisant l'Input/Output au niveau matériel.

Transaction

Signification Sémantique: Une Transaction est une unité logique de traitement atomique regroupant une ou plusieurs instructions SQL, conçue pour faire passer le Database d'un état de cohérence validé à un autre état de cohérence validé. **Simplified Version**: Une séquence d'actions groupées qui doivent absolument toutes réussir ensemble. Si une seule action échoue, l'ensemble du groupe est annulé comme si rien ne s'était passé. **Frontières Théoriques**: La Transaction n'est pas une requête individuelle, mais un bloc d'exécution. La Transaction s'achève obligatoirement par un Commit ou un Rollback. **Rôle Architectural**: La Transaction est le véhicule principal par lequel le Database Management System garantit l'intégrité concurrentielle et la récupération après panne selon le paradigme ACID. **Illustration Structurelle**:

```sql
BEGIN TRANSACTION;
UPDATE Account SET Balance = Balance - 100 WHERE AccountID = 1;
UPDATE Account SET Balance = Balance + 100 WHERE AccountID = 2;
COMMIT;
```

Valeur NULL

Signification Sémantique: La Null Value est un marqueur sémantique spécial dans le Relational Model indiquant l'absence structurelle, l'inconnue ou la non-applicabilité d'une donnée pour un Attribute spécifique. **Simplified Version**: Un marqueur formel signifiant qu'une information est manquante, inconnue, ou qu'elle n'a pas de sens pour cet élément particulier, ce qui est très différent d'un zéro ou d'un texte vide. **Frontières Théoriques**: La Null Value n'est pas le chiffre zéro ni une chaîne de caractères vide. La Null Value n'obéit pas à l'algèbre booléenne standard, générant une logique ternaire. **Rôle Architectural**: L'autorisation de la Null Value dicte la structure du Physical Schema. Son interdiction absolue est requise pour tout Identifying Attribute ou Primary Key afin de préserver l'Integrity Constraint.

Vue

Signification Sémantique: Une View est une relation virtuelle et dynamique dont le contenu est instancié à la volée par l'exécution d'une requête SQL persistante stockée dans le Data Dictionary. **Simplified Version**: Un tableau virtuel construit sur mesure qui n'enregistre aucune donnée lui-même, mais affiche en direct un filtre ou un calcul basé sur d'autres tableaux réels. **Frontières Théoriques**: La View standard ne stocke aucune Data physique sur le disque. La modification des Data au travers d'une View est structurellement limitée. **Rôle Architectural**: La View fournit la fondation de l'External Schema, garantissant la sécurité en masquant les Column sensibles et simplifiant l'accès pour la Business Logic en encapsulant des Join complexes. **Illustration Structurelle**:

```sql
CREATE VIEW ActiveUsers AS
SELECT UserID, Username
FROM User
WHERE IsActive = 1;
```

Weak Entity Class

Signification Sémantique: Une Weak Entity Class est une classe d'entités qui est sémantiquement incapable d'identifier ses instances de manière autonome et dépend de l'existence d'une Strong Entity Class propriétaire pour sa propre identification. **Simplified Version**: Une catégorie d'informations dont les éléments ne peuvent pas exister ni être reconnus sans être rattachés de manière stricte à un élément principal appartenant à une autre catégorie. **Frontières Théoriques**: La Weak Entity Class n'a pas de Primary Key absolue propre. La Weak Entity Class possède uniquement un Partial Key. **Rôle Architectural**: Dans le Relational Schema, la Weak Entity Class est implémentée en combinant obligatoirement la Foreign Key de la Strong Entity Class hôte avec son propre Partial Key pour forger un Composite Primary Key robuste.

Accès concurrents

Signification Sémantique: Le Concurrent Access désigne l'exécution simultanée de multiples Transaction par différents processus ou utilisateurs tentant de lire ou de modifier le même sous-ensemble de Data au sein du Database. **Simplified Version**: La situation où plusieurs utilisateurs ou programmes essaient de lire ou de modifier les mêmes informations exactement au même moment. **Frontières Théoriques**: Le Concurrent Access n'est pas un défaut du système mais une exigence de performance. Le Concurrent Access génère des anomalies transactionnelles si l'Isolation n'est pas rigoureusement appliquée. **Rôle Architectural**: La gestion du Concurrent Access incombe au Relational Database Management System via des mécanismes de Locking (optimiste ou pessimiste) pour maintenir la consistance des données.

Association (Binaire, Ternaire, Récursive, Cyclique)

Signification Sémantique: L'Association définit la topologie relationnelle entre les Entity Class. Elle peut être Binary (deux classes), Ternary (trois classes indissociables), Recursive (une classe liée à elle-même) ou Cyclic (une boucle de dépendances entre multiples classes). **Simplified Version**: Les différentes formes géométriques que peuvent prendre les liens entre les catégories d'informations, allant du simple duo jusqu'à des boucles complexes ou des connexions à trois éléments. **Frontières Théoriques**: L'Association structurelle ne contient aucune Data métier propre en dehors des Foreign Key nécessaires à la résolution de la jointure, sauf dans le cas d'une Associative Entity. **Rôle Architectural**: Le type d'Association dicte directement l'architecture du Relational Schema, déterminant si l'implémentation requiert le simple ajout d'une Foreign Key ou la création d'une Table de jonction dédiée.

Cardinalité (Totale, Partielle, Exclusive, Overlapping)

Signification Sémantique: La Cardinality Constraint définit les règles de participation d'une Entity Class. Elle spécifie si la participation est Total (obligatoire) ou Partial (optionnelle), et dans le cas de l'Inheritance, si la spécialisation est Exclusive (disjointe) ou Overlapping (superposée). **Simplified Version**: Les règles mathématiques strictes définissant si un élément doit obligatoirement être lié à un autre, s'il peut exister seul, et s'il a le droit d'appartenir à plusieurs sous-catégories en même temps. **Frontières Théoriques**: La

Cardinality Constraint ne s'applique qu'à la définition structurelle de l'Entity-Relationship Model et ne gère pas la Business Logic dynamique de l'application. **Rôle Architectural**: La Cardinality Constraint impose la création de contraintes NOT NULL, de Check Constraint, ou de Trigger complexes au niveau du Physical Schema pour empêcher toute insertion de Data invalide.

CODASYL

Signification Sémantique: CODASYL (Conference on Data Systems Languages) désigne le standard historique définissant le Network Model pour la gestion des Database, précédant l'avènement du Relational Model. **Simplified Version**: Une norme très ancienne utilisée avant l'invention des tableaux relationnels, organisant les informations sous forme de réseau où chaque donnée pouvait être directement connectée à n'importe quelle autre. **Frontières Théoriques**: CODASYL ne repose pas sur l'algèbre relationnelle. Le Network Model permet de multiples parents par Record, contrairement au Hierarchical Model, mais souffre d'un couplage fort avec le Physical Schema. **Rôle Architectural**: CODASYL nécessitait l'utilisation de pointeurs physiques complexes, rendant le Data Manipulation Language extrêmement procédural et dépendant de la topologie de stockage sur disque.

Couche logique / Schéma de communauté

Signification Sémantique: Le Logical Layer, ou Community Schema, est l'abstraction intermédiaire unifiée décrivant l'ensemble des Entity Class, des Attribute et des Association pour toute l'organisation, normalisée selon le Relational Model indépendamment des contraintes matérielles. **Simplified Version**: Le plan technique global et partagé de toute l'entreprise, transformant les concepts abstraits en tableaux et colonnes formels prêts à être programmés. **Frontières Théoriques**: Le Logical Layer ignore les Index d'optimisation et les détails d'allocation physique du Storage Engine. Le Logical Layer fusionne tous les External Schema en une unique vérité normalisée. **Rôle Architectural**: Le Logical Layer constitue la base mathématique stable sur laquelle le Data Definition Language est généré avant d'être instancié sur un Database Management System spécifique.

Couche physique / Couche interne

Signification Sémantique: Le Physical Layer, ou Internal Schema, représente l'implémentation de plus bas niveau du Database, définissant les structures de données réelles, le partitionnement, les méthodes d'accès et les Index sur le support de stockage matériel. **Simplified Version**: La configuration technique ultime qui dicte exactement comment, où, et sous quel format précis les fichiers sont inscrits et organisés sur le disque dur. **Frontières Théoriques**: Le Physical Layer ne modifie jamais la sémantique ni la logique relationnelle du Logical Layer. Le Physical Layer est totalement dépendant du Database Management System et du matériel choisi. **Rôle Architectural**: Le Physical Layer optimise drastiquement les performances d'Input/Output et l'exécution du Data Manipulation Language, isolant cette complexité des applications clientes grâce à l'indépendance physique des données.

Entity Class (Faible, Forte)

Signification Sémantique: Une Strong Entity Class possède un Identifying Attribute propre permettant l'identification autonome de ses instances. Inversement, une Weak Entity Class dépend existentiellement de la Primary Key d'une Strong Entity Class propriétaire pour forger son identité. **Simplified Version**: Une catégorie d'informations indépendante possède son propre identifiant unique, tandis qu'une catégorie dépendante ne peut exister et être identifiée qu'en s'accrochant à l'identifiant d'une catégorie principale.

Frontières Théoriques: La Weak Entity Class n'est pas simplement une Table avec une Foreign Key classique; son existence même est subordonnée à l'entité forte. **Rôle Architectural**: L'instanciation de la Weak Entity Class dans le Relational Schema requiert obligatoirement la création d'une Composite Primary Key incluant la Foreign Key pointant vers la Strong Entity Class.

Héritage / Héritage multiple

Signification Sémantique: L'Inheritance est le mécanisme où une Subclass hérite des Attribute d'une Superclass (Relation Is-A). Le Multiple Inheritance survient lorsqu'une Subclass dérive simultanément de plusieurs Superclass orthogonales. **Simplified Version**: Le principe par lequel une sous-catégorie reçoit automatiquement toutes les caractéristiques de sa ou ses catégories parentes, permettant de ne pas réécrire les mêmes informations plusieurs fois. **Frontières Théoriques**: Le Multiple Inheritance introduit des complexités de résolution d'ambiguïté (problème du diamant). Le Relational Model pur ne supporte pas nativement l'Inheritance orienté objet. **Rôle Architectural**: L'Inheritance requiert l'implémentation de design patterns spécifiques dans le Physical Schema, tels que le Class Table Inheritance ou le Single Table Inheritance, souvent via un Discriminator Attribute.

Indépendance des données

Signification Sémantique: La Data Independence est le principe architectural séparant fondamentalement la définition des Data (Logical Layer et Physical Layer) de la logique des applications métier qui les consomment (External Layer). **Simplified Version**: Le principe de conception fondamental garantissant que l'on peut modifier l'organisation des fichiers ou des tableaux sans jamais casser les programmes qui les utilisent. **Frontières Théoriques**: L'indépendance logique protège contre les changements du Community Schema, tandis que l'indépendance physique protège contre les changements de l'Internal Schema. **Rôle Architectural**: La Data Independence est assurée par le Database Management System, permettant aux administrateurs de réorganiser les disques ou d'ajouter des Index sans devoir recompiler le code source des applications clientes.

Langage déclaratif

Signification Sémantique: Un Declarative Language, comme SQL, est un paradigme syntaxique où l'utilisateur spécifie uniquement le résultat final attendu (le quoi) sans programmer explicitement les étapes d'exécution séquentielle algorithmique (le comment). **Simplified Version**: Un langage de commande où l'on décrit simplement le résultat que l'on souhaite obtenir, laissant le moteur intelligent du système décider lui-même de la meilleure façon de faire le travail. **Frontières Théoriques**: Un Declarative Language n'inclut pas de boucles (FOR/WHILE) ni de structures de contrôle de flux conditionnel par défaut, n'étant pas généraliste ni Turing-complet dans sa forme stricte. **Rôle Architectural**: Le Declarative Language délègue l'entière responsabilité de l'optimisation des chemins d'accès au Query Optimizer intégré dans le Database Management System.

Langage procédural

Signification Sémantique: Un Procedural Language est un paradigme de programmation impérative où le développeur doit définir explicitement la séquence exacte des instructions de contrôle de flux, des boucles et des branchements pour manipuler les Data. **Simplified Version**: Un langage de programmation classique où le développeur doit dicter à l'ordinateur chaque étape, chaque boucle et chaque condition, l'une après l'autre, pour obtenir le résultat voulu. **Frontières Théoriques**: Le Procedural Language s'oppose au

Declarative Language. Les langages procéduraux étendent souvent SQL (comme PL/SQL ou T-SQL) pour permettre une Business Logic complexe au sein même de la base. **Rôle Architectural**: Le Procedural Language est utilisé pour coder des Stored Procedure, des Trigger et des User-Defined Function, encapsulant la logique de traitement transactionnel complexe directement sur le serveur Database. **Illustration Structurelle**:

```
CREATE PROCEDURE UpdateSalaries()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    -- Boucles et curseurs procéduraux omis pour concision
    UPDATE Employee SET Salary = Salary * 1.05;
END;
```

Modèle orienté objets

Signification Sémantique: L'Object-Oriented Model est un paradigme de structuration où les Data sont encapsulées avec leurs comportements algorithmiques au sein de classes complexes, supportant l'Inheritance et le polymorphisme, plutôt que sous forme de relations tabulaires plates. **Simplified Version**: Une méthode de structuration où l'on range les informations sous forme de véritables objets informatiques complexes qui possèdent non seulement des caractéristiques, mais aussi leurs propres comportements. **Frontières Théoriques**: L'Object-Oriented Model ne repose pas sur l'algèbre relationnelle. L'Object-Oriented Model lie intrinsèquement les Data à la Business Logic de l'application. **Rôle Architectural**: Ce modèle a mené au développement des Object-Oriented Database Management Systems (OODBMS) et a nécessité la création d'Object-Relational Mapping (ORM) pour faire le pont avec les bases de données relationnelles classiques.

Modèle relationnel

Signification Sémantique: Le Relational Model est le paradigme mathématique fondamental proposé par E.F. Codd, structurant la totalité des Data sous forme de n-uplets (Record) au sein de relations bidimensionnelles plates (Table) gouvernées par l'algèbre relationnelle. **Simplified Version**: Le modèle mathématique standard et dominant qui organise strictement toutes les informations sous forme de grilles et de tableaux bidimensionnels interconnectés. **Frontières Théoriques**: Le Relational Model interdit par définition les attributs composites ou les listes imbriquées à l'intérieur d'une seule colonne (principe de la Première Forme Normale). **Rôle Architectural**: Le Relational Model fournit le socle théorique absolu sur lequel reposent tous les Relational Database Management Systems modernes (RDBMS) et le standard syntaxique SQL.

Redondance d'information

Signification Sémantique: La Data Redundancy qualifie l'anomalie architecturale où une même information sémantique est physiquement dupliquée et stockée dans de multiples Table ou Column non dérivées au sein du Relational Schema. **Simplified Version**: Le défaut de conception consistant à écrire et enregistrer plusieurs fois la même information à des endroits différents, gaspillant de l'espace et risquant de générer des contradictions. **Frontières Théoriques**: La Data Redundancy ne doit pas être confondue avec la réplication des disques physiques (qui est une stratégie de sauvegarde). La Data Redundancy est une erreur conceptuelle au niveau du Logical Layer. **Rôle Architectural**: La Data Redundancy provoque des

anomalies d'insertion, de modification et de suppression. Elle est systématiquement éliminée lors de l'application du processus rigoureux de Database Normalization.

Roll back

Signification Sémantique: Le Rollback est l'opération transactionnelle de sécurité qui annule l'intégralité des modifications du Data Manipulation Language non validées effectuées depuis le début de la Transaction, restaurant le Database à son état consistant précédent. **Simplified Version**: Le bouton d'annulation d'urgence qui efface immédiatement toutes les modifications en cours et remet la base de données exactement dans l'état où elle était avant que le problème ne survienne. **Frontières Théoriques**: Le Rollback ne peut annuler une Transaction qui a déjà été scellée par un Commit. Le Rollback n'est possible que si la propriété d'Atomicity du paradigme ACID est activée. **Rôle Architectural**: Le Rollback est orchestré par le Transaction Log du Database Management System, permettant la reprise sur erreur (Error Recovery) face aux pannes matérielles ou aux violations d'Integrity Constraint. **Illustration Structurelle**:

```sql
BEGIN TRANSACTION;
DELETE FROM ImportantTable WHERE ID = 42;
-- Une erreur critique est détectée
ROLLBACK TRANSACTION;
```

UML diagramme de classes

Signification Sémantique: L'UML Class Diagram est une notation visuelle standardisée du Unified Modeling Language utilisée lors du Conceptual Design pour représenter l'architecture statique d'un système via ses Entity Class, Attribute et Association. **Simplified Version**: Le dessin technique standardisé utilisé par les ingénieurs pour schématiser sous forme de boîtes et de flèches comment les différents objets du système seront programmés et liés entre eux. **Frontières Théoriques**: L'UML Class Diagram est orienté objet et n'est pas strictement équivalent à un Entity-Relationship Diagram relationnel pur. L'UML Class Diagram modélise également des méthodes applicatives que le Database ignorera. **Rôle Architectural**: L'UML Class Diagram sert de Blueprint indépendant de la plateforme, permettant de générer automatiquement à la fois le code source des applications et les scripts Data Definition Language pour le Database.